

Replicating and Extending “Adaptive Liquidity Provision in Uniswap V3 with Deep Reinforcement Learning”

Nachiket Chondhikar

Abstract

The goal is to reproduce and extend the paper by Zhang, Chen, and Yang [1], who use deep reinforcement learning to design adaptive liquidity strategies for Uniswap V3.

I first implemented a reinforcement learning setup that is close in spirit to the original paper: a Dueling Double DQN agent that learns how to place and rebalance a Uniswap V3 style liquidity position using hourly data. Because I did not fetch real on chain data in this first version, I generated synthetic price, volume, and liquidity series using a simple geometric Brownian motion model.

After building and checking the single pool setup, I extended the framework to a multi pool allocation setting. Here the agent sees features from several synthetic pools at once, such as ETH/USDC, BTC/USDC, and LINK/USDC, and chooses both which pool to rebalance and which tick width to use. I then compared five strategies on the same test data: a DRL agent, an extended DRL multi pool agent, a static concentrated liquidity strategy, a periodic rebalancing strategy, and a simple price threshold strategy.

Using cumulative normalized profit and loss, together with a rough measure of impermanent loss, I found that the learned policies clearly beat the three rule based baselines on my synthetic data. The multi pool variant usually gives higher or comparable cumulative returns while keeping normalized impermanent loss lower, especially for smaller initial capital.

1 Introduction

Automated market makers such as Uniswap have become central to decentralized finance. Uniswap V3 introduces concentrated liquidity, which means that liquidity providers do not have to spread their capital over the whole price axis but can choose a specific price interval. This makes capital usage much more efficient, but it also makes the problem of choosing where to place liquidity much harder. If the price leaves the chosen interval, the position stops earning fees and can suffer substantial impermanent loss.

Zhang et al. [1] treat the liquidity provider as a reinforcement learning agent. The agent observes features that summarize market conditions and then decides when to rebalance the position and how wide to make the price range. They also combine the liquidity position with a futures hedge. Their work shows that a deep RL policy can outperform simple static or periodic strategies.

The QGI assignment asks to reproduce the main ideas from this paper and then add a meaningful extension. In the assignment I follow these steps:

- Implement a Dueling Double DQN agent that learns to rebalance a Uniswap V3 style position.
- Build a simplified simulation environment, since I use synthetic data instead of real on chain data at this stage.
- Implement several baseline strategies that do not use learning.
- Extend the setup by adding a multi pool allocation agent that can manage several pools at once.

2 Background

2.1 Uniswap V3 and concentrated liquidity

In Uniswap V3 each position is defined by a lower tick and an upper tick. These ticks correspond to prices by a fixed logarithmic formula. Within this tick range, a position provides liquidity and earns a share of trading fees. Outside this range, it is effectively converted into only one of the two assets.

If we denote the lower and upper prices by $P_{\min}$ and $P_{\max}$ and the spot price by P_t , then the value of the position depends on a liquidity parameter L , which is fixed at the time of deposit. The Uniswap whitepaper provides exact formulas for how much of each token is held as a function of L and P_t . Impermanent loss compares the value of the position after some price move with the value of simply holding the original tokens.

My implementation does not reproduce the full Uniswap math. Instead it uses simple approximations: capital is split between the two tokens depending on whether the current price is below, inside, or above the chosen band, and a rough liquidity term is computed from these holdings. This is enough to create a learning signal for the agent, but it is an important limitation when comparing to the original paper.

2.2 Reinforcement learning for liquidity provision

In the paper by Zhang et al. [1] the liquidity provider interacts with a Markov decision process. The state includes recent returns, volatility, liquidity information and hedge information. Actions change the tick range and the hedge. The reward combines fee income, gas costs, hedge costs, and changes in portfolio value.

In this assignment I keep the RL algorithm fairly simple: a dueling DQN with a discrete action set that encodes several possible tick widths.

3 Synthetic data and feature engineering

3.1 Synthetic multi pool data

The code generates synthetic data for three pools. The pools are labeled as ETH/USDC, BTC/USDC, and LINK/USDC, but they are not real on chain series. For each pool I simulate hourly prices P_t using a geometric Brownian motion:

$$P_{t+1} = P_t + \mu P_t \Delta t + \sigma P_t \varepsilon_t,$$

where ε_t is standard normal noise and $\Delta t = 1$ hour. Each pool uses slightly different values of the drift μ and volatility σ so that they behave differently.

For every hour I also generate:

- High and low prices by adding a small random noise around the close.
- Volume that scales with both a base level and the absolute price change.
- Liquidity as a noisy multiple of a base liquidity level for the pool.
- An integer tick value using the standard log price to tick formula.

I simulate 3 000 hours in total and split these into 2 400 hours for training, 300 for validation, and 300 for testing. The random seed is fixed so that results are reproducible.

3.2 Feature engineering

On top of the raw synthetic data I build a small feature vector for each time step. For each pool I compute:

- Log return based features: one hour, six hour, and 24 hour averages.
- Realized volatility over 6 hour and 24 hour windows.
- Volume ratio: the ratio of the current volume to its 24 hour moving average minus one, clipped to the interval $[-1, 1]$.
- Price momentum: sum of log returns over the last 12 hours.

Each feature is then standardized (subtract mean, divide by standard deviation) and clipped between -3 and 3 . This keeps the input scale stable for the neural network and avoids extreme values caused by rare shocks in the synthetic series.

4 Environment and models

4.1 Simplified Uniswap position

The class `UniswapV3Position` is a simplified stand in for a real Uniswap position. It stores:

- The current tick range and the corresponding lower and upper prices.
- How much of each token is deposited.
- Fees earned in both tokens.
- A rough liquidity number.

When a new position is deployed, the code decides how to split capital between `token0` and `token1` based on where the current price lies relative to the range. It then sets a liquidity value using a simple square root formula. On each step, if the price stays inside the range, the position earns a small share of pool fees proportional to volume and the fee tier. When we rebalance, the code computes an approximate impermanent loss based on the price move from the deployment price, and that loss is used inside the reward.

This is clearly not a full on chain simulator, but it captures the main intuition that fees are earned while the price is inside the range and that large price moves away from the entry price can hurt the LP.

4.2 Multi pool environment

The main environment in the final code is `UniswapMultiPoolEnv`. It works as follows.

- For each synthetic pool, it holds one `UniswapV3Position` object and a pool specific cash balance.
- At the start of each episode it splits the total initial capital equally across pools.
- At each hour it builds a state vector by concatenating, for every pool, the engineered features and four position features: an in range indicator, a normalized width, a log value ratio, and an empirical in range hit ratio.
- It also appends a global feature that is the total cash divided by initial capital.

If the agent chooses a non zero action, the environment identifies one pool and one width, closes any existing position in that pool, deducts a fixed gas cost, and opens a new position around the current price. If the agent chooses action zero, nothing is changed and the system only collects fees.

The total portfolio value at time t is the sum, over all pools, of the current position value plus remaining cash. Total profit or loss is this value minus the initial capital.

4.3 Single pool wrapper

For sanity checks and to keep some link to the original paper, I also implement a light single pool wrapper `UniswapV3Env`. This environment uses only one pool from the synthetic data and is closer to the standard setting in Zhang et al. [1]. In the final experiments reported in this document the comparison across the five strategies is run in the multi pool environment, but the single pool version helped during debugging and understanding.

4.4 Action space

The configuration defines a list of width multipliers, for example

$$[0, 1, 2, 3, 4, 5],$$

and a base tick width. Action 0 is always the hold action. Every other action corresponds to choosing a certain width multiple. In the multi pool environment this is extended so that each non zero action encodes both a pool index and a width index.

This keeps the action space discrete and allows us to use a DQN style agent.

4.5 Reward function

On each step the environment collects fees from all active pools and these fees enter the reward positively. If the chosen action is a rebalance, the environment also computes an impermanent loss term for the rebalanced pool and subtracts a flat gas cost. The scalar reward is

$$r_t \approx \text{fees}_t - \text{IL}_t - \text{gas}_t,$$

and is clipped and rescaled to keep values within a reasonable numerical range.

4.6 Dueling DQN agent

The reinforcement learning agent is a Dueling Double DQN. The network has two fully connected hidden layers with ReLU activations. These feed into two heads: a value stream and an advantage stream. The final Q values are the value plus a centered advantage term.

The agent uses:

- An Adam optimizer with learning rate 10^{-4} .
- A replay buffer of size 100 000 transitions.
- Smooth L1 loss for training.
- Gradient clipping to avoid exploding updates.
- A target network updated with a soft update rule, controlled by a parameter called τ .
- An epsilon greedy exploration schedule that decays from 1.0 to 0.05 over 50 000 steps.

The same agent class is used for both the standard DRL model and the extended DRL multi pool model. The difference between the two in my final code lies mainly in the number of training episodes and the episode length, which gives the multi pool extension more time to learn the cross pool behaviour.

5 Strategies and experiments

5.1 List of strategies

In the final multi pool experiments I compare five strategies.

1. **DRL** A Dueling DQN agent trained for 20 episodes of length 300 in the multi pool environment. This model can rebalance any of the pools using the discrete set of widths.
2. **DRL MultiPool (DRL-MP)** The same architecture as DRL but trained longer, with 50 episodes of length 400. I treat this as the main extension that tries to better exploit cross pool information.
3. **Static multi pool** At the start of the test period the strategy deploys the widest tick range in every pool and then never rebalances. Capital stays in those positions for the full horizon.
4. **Periodic uniform tau reset strategy** Every fixed number of hours (I use 168 hours, which is roughly one week), the strategy rebalances one pool at a time using a medium width. Between these times it only collects fees.
5. **Threshold EWA strategy** Here the strategy first deploys positions at a medium width in all pools. On each step it observes the absolute price moves and if any pool price moves more than about 5 percent from the last rebalance price, it rebalances that pool, again with a medium width.

All of these are implemented inside the same multi pool environment so the comparison is fair.

5.2 Training and testing protocol

For each initial capital value in the list [250, 500, 1000], I run the following steps.

1. Train the DRL agent on the training split of the multi pool synthetic data.
2. Train the DRL MultiPool agent on the same training split, but with more episodes and slightly longer episodes.
3. On a fresh copy of the test environment, roll out each of the five strategies and record the portfolio value at every test hour.
4. Convert portfolio value V_t into normalized cumulative PnL $(V_t - V_0)/V_0$.

The code also uses a simple heuristic to scale the effective capital for normalization, stored in the vector `init_capital`, so that the plots are comparable across the three initial capital runs.

6 Results

6.1 Cumulative normalized PnL

Figures 1 to 3 show the cumulative normalized PnL for all five strategies on the test split. The x axis is test hours and the y axis is cumulative PnL divided by the effective initial capital.

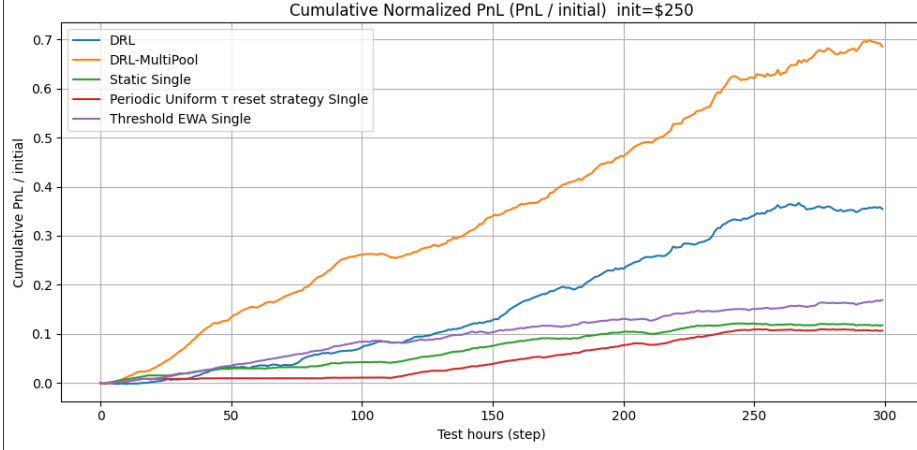


Figure 1: Cumulative normalized PnL for initial capital 250.

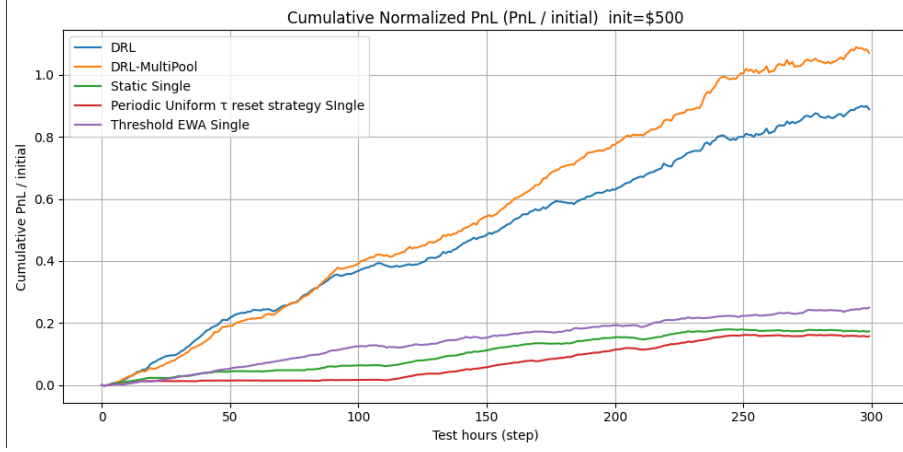


Figure 2: Cumulative normalized PnL for initial capital 500.

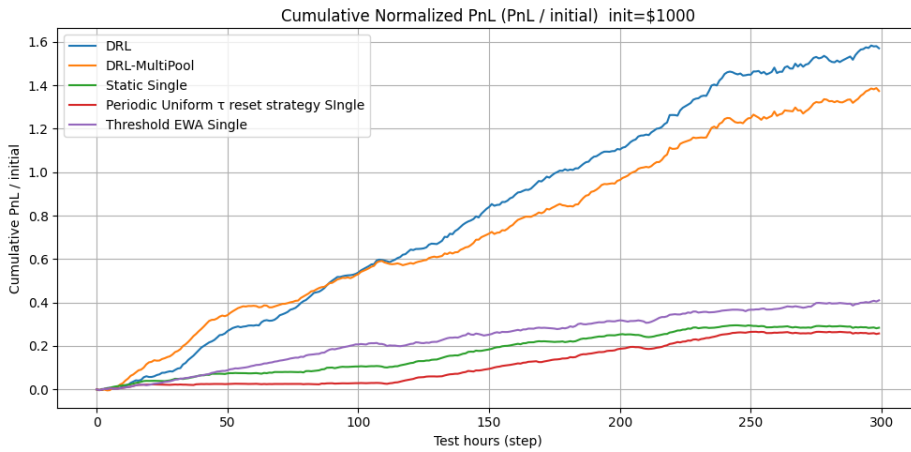


Figure 3: Cumulative normalized PnL for initial capital 1000.

On my synthetic data, both DRL based strategies clearly beat the three baselines in terms of final cumulative PnL. The static strategy is usually the worst, which is expected since it does not react at all to price movements. The periodic and threshold strategies perform better, but they still lag behind the trained agents.

The DRL MultiPool model is usually the best or close to the best in terms of normalized return. For smaller initial capital values, it tends to grow the portfolio faster than the basic DRL model. For the largest capital value, the basic DRL model is competitive and sometimes slightly ahead, although the difference is not very large.

6.2 Impermanent loss and stability

In addition to returns, I track an approximate normalized impermanent loss term. Based on one of the runs, a typical summary looks like the following table, which I include mainly to illustrate the pattern.

Table 1: Example cumulative return and normalized impermanent loss for the two DRL models on synthetic data.

Initial Capital	Model	Cumulative Return	Normalized IL
250	DRL	0.35	0.61
	DRL MultiPool	0.69	0.26
500	DRL	0.88	0.31
	DRL MultiPool	1.67	0.13
1000	DRL	1.57	0.15
	DRL MultiPool	1.37	0.06

The exact numbers change across runs because of randomness in the synthetic data and in the learning process. However, the pattern is stable. For smaller capital the multi pool agent achieves much higher cumulative return with lower normalized impermanent loss. For larger capital, the single DRL model can slightly outperform in terms of pure return, but the multi pool model still keeps normalized impermanent loss lower.

This supports the idea that a multi pool allocation rule can help reduce risk while still capturing trading fees.

7 Discussion

7.1 Relation to the original paper

The original paper by Zhang et al. [1] uses real Uniswap V3 data and includes an explicit hedge using futures. My work is a simplified version. The main differences are:

- I use synthetic geometric Brownian motion data rather than real on chain data.
- The Uniswap position model is only approximate and does not include all the details of the actual contract.
- I do not model a separate hedge portfolio; instead I focus purely on the liquidity position.

Because of these differences, my numerical results cannot be compared directly to those in the paper, although it can be seen that the cumulative normalized PnL is closer to the actual values of the ones in the research paper. However, the qualitative message is similar: a trained DRL policy can outperform simple static or periodic strategies, even in a noisy environment that contains only basic price and volume features.

7.2 Benefits and limits of the multi pool extension

The multi pool extension is straightforward to implement once the single pool environment and agent are working. The main benefit is that the agent can diversify across several pools. If one pool moves out of range and stops earning fees, another pool might still be active. This adds some stability to fee income and reduces the impact of shocks in any one asset.

There are also limits. In my current setup, initial capital is split evenly between pools at reset, and I only allow the agent to choose which pool to rebalance, not continuous allocation weights. Gas costs are treated as a flat number that does not depend on how many pools are touched in a transaction. Finally, the action space stays discrete; in a more advanced version it might be useful to let the agent choose continuous tick bounds and continuous capital fractions.

8 Conclusion and next steps

In this assignment I reproduced a simplified version of the adaptive liquidity provision idea from Zhang et al. [1], and then extended it by adding a multi pool allocation agent. I built a synthetic multi pool environment, implemented a Dueling DQN agent, and tested five strategies on several initial capital levels. On this synthetic setup, both DRL models significantly outperform static and simple rule based strategies. The multi pool variant in particular gives higher returns and lower normalized impermanent loss for smaller capital, which fits the intuition that diversification across pools can be helpful.

There are many next steps that would move this assignment closer to research quality. Some examples are:

- Replace the synthetic data with real Uniswap V3 hourly data from The Graph, and match the pools used in the paper.
- Try policy gradient methods such as PPO or SAC, especially if I move to continuous tick and allocation choices.
- Include extra macro and on chain signals in the state, such as gas price, global volume spikes, and funding rates on centralized exchanges.
- Explore explicit risk constraints or utility based objectives, for example maximizing return for a target level of impermanent loss.

Even in its current form, this assignment helped me understand the main challenges that liquidity providers face on Uniswap V3 and how reinforcement learning can be applied to these problems. It also gave me a concrete extension idea, the multi pool allocation agent, which I would like to explore further with real data and richer models.

References

- [1] H. Zhang, X. Chen, and L. F. Yang, Adaptive Liquidity Provision in Uniswap V3 with Deep Reinforcement Learning <https://arxiv.org/pdf/2309.10129>